DEEP Q LEARNING

Reinforcement Learning Project 3 Report

Hucey, Chad

Overview

In this project, Deep Q-learning was used to train an agent to play Breakout, an Atari game. Figure 1 shows a screenshot of the game.



Figure 1: Breakout[i]

The agent operates in the game environment. Each state is considered to be a stack of 4 still images, and as a result, it is infeasible to assume a finite state space. To address the problem, this project implements Deep Q-Learning to approximate the value function.

Approximating the Value Function

As mentioned previously, assuming a finite state space is infeasible. So, an approximation to the value function is made. This approximation was made using a convolutional neural network (CNN) with the state as the input and the approximated value function as the output as shown in Figure 2.



Figure 2: Non-linear Value Function Approximation

Convolutional Neural Network

Figure 3 shows the CNN architecture. It consists of three convolutional layers followed by two linear layers. The CNN architecture was chosen as it is suitable for extracting patterns from image data.

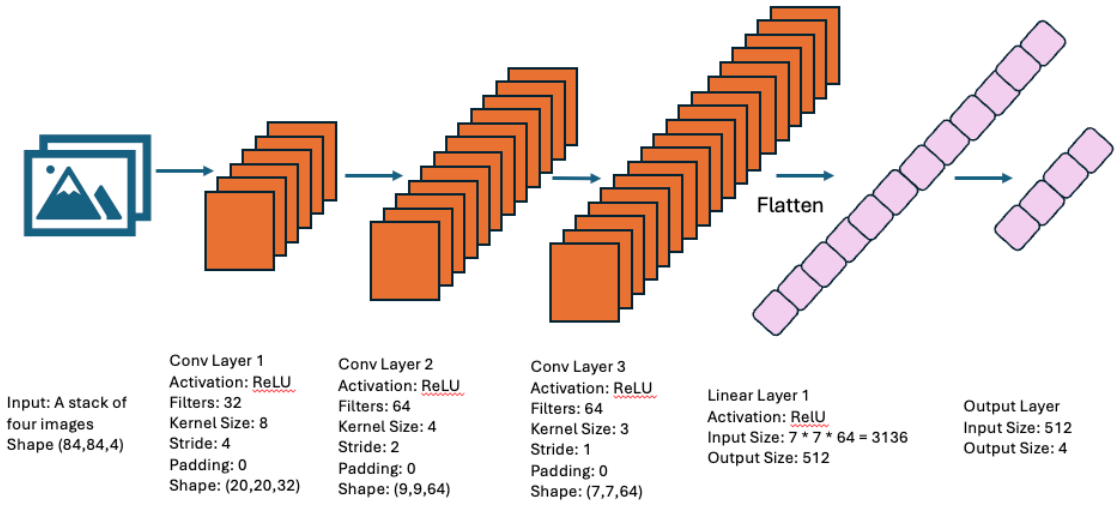


Figure 3: CNN Architecture

All layers except the output layer use ReLU as the activation function to add non-linearity to the value function estimator. The output layer uses no activation function, as the Huber Loss is used as the loss criterion.

Training the Agent

With the CNN initialized, the agent was trained using Deep Q-Learning. The process used to train the agent is as follows:

1. Initialized an updatable CNN and a target CNN, both with the architecture mentioned above. Doing this helped to stabilize training
2. Explored the environment and gathered episodes:
    a. Looped through each time step in each episode:
        i. Given a state, took an action using a greedy policy
        ii. Obtained the reward and next state
        iii. Added the timestep of the form (state, action, reward, next_state, done) to a buffer
        iv. Sampled 32 samples from the buffer
        v. Calculated target $y = r_t + \max_a Q_{target}(s_{t+1}, a)$
        vi. Updated the weights of the updatable CNN

vii. Periodically updated the weights of the target CNN every 500 timesteps
viii. Saved an updatable CNN model every time the average reward after 30 lives increased

b. After 5 lives were lost, reset the environment and start a new episode
c. Trained for 5,000 episodes

3. Plotted the training curve

As part of the training process, the Huber loss was used as the loss criterion, and Adam was used as the optimizer. Additionally, the gradients were clipped to between -1 and 1.

After training the agent, Figure 4 shows the training curve, plotting time steps vs. average reward after losing 30 lives.

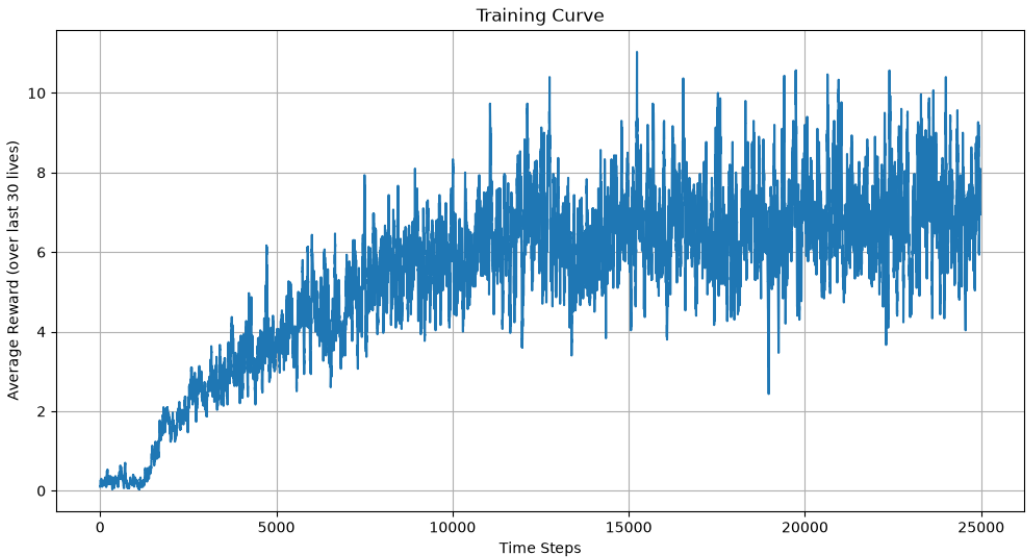


Figure 4: Training Curve

Test Results

The trained model was evaluated to determine the average reward it earned after 100 lives. Figure 5 shows the results of this evaluation.

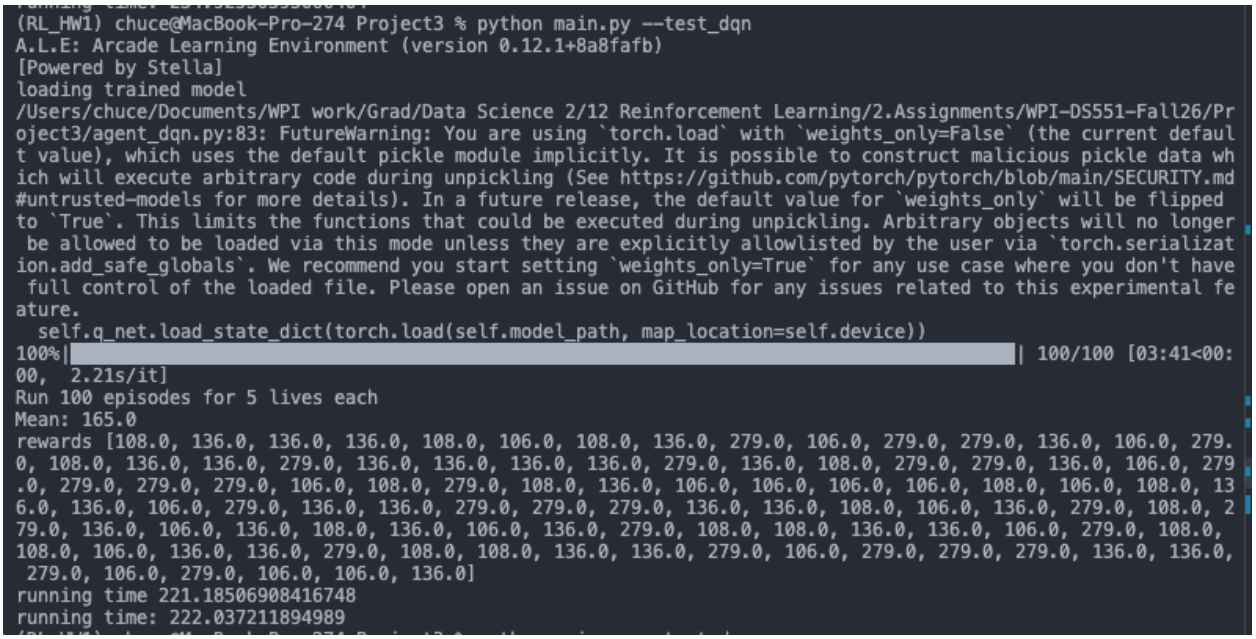


Figure 5: Test Results

The agent achieved an average reward of 165 across 100 lives, thus surpassing the threshold of 40.

Although the agent surpassed the goal of an average reward of 40, it could still be improved. Techniques such as Double DQN or Deuling DQN could have been utilized to train a better model. In future work, these techniques will be used.

[i] Obtained From *ALE documentation*. (2023). Farama.Org. https://ale.farama.org/environments/breakout/